

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 4

Binární halda

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>



ZS 2025/2026, 13. října 2025

(Verze dokumentu: 14. 12. 2025 19:21)

Online verze problému zpracování úkolů s prioritami

- Úkoly chodí průběžně během zpracovávání předchozích.
- Obecně online verze není lehčí než offline verze.

Přeformulovaný problém

Chceme navrhnout datovou strukturu pro reprezentaci **dynamicky se měnící množiny porovnatelných hodnot** (např. priorit úkolů), nad kterou bude možné **efektivně** provádět následující **operace**:

- vložení nové hodnoty (**Insert**),
- nalezení momentálního minima množiny (**FindMin**) a
- odstranění tohoto minima z množiny (**ExtractMin**).

Řešení online verze pomocí neseřazeného pole

Časová složitost:

- Vložení bude trvat čas $O(1)$ (připojíme na konec).
- Nalezení minima čas $O(n)$ (průchod celým polem).
- Odstranění minima $O(n)$ (bude třeba pole „sešoupnout“, aby v něm nebyla „díra“).

K zamyšlení:

- Jak se změní složitosti, když jako datovou strukturu použijeme spojový seznam?
- Jak se změní složitosti, když jako datovou strukturu použijeme pole, které budeme průběžně udržovat seřazené?

Binární minimová halda

- Jednoduchá, ale velice užitečná datová struktura.
- Uchovává navzájem porovnatelné prvky (pro jednoduchost celá čísla), kterým budeme nadále říkat **klíče**.
- Podporuje tyto operace:

Operace	Čas. slož.	Popis
<code>HeapInsert(H, k)</code>	$O(\log n)$	Vloží nový klíč k do haldy H .
<code>HeapFindMin(H)</code>	$O(1)$	Najde a vrátí minimum ze všech prvků v haldě H .
<code>HeapExtractMin(H)</code>	$O(\log n)$	Odstraní minimum z haldy H a vrátí ho.

- Analogicky lze definovat **binární maximovou haldu**.

Graf bez vrcholů

- V teorii grafů je graf bez vrcholů zvláštním případem, který je často třeba z obecně platných tvrzení vylučovat.
- Proto jej v takových tvrzeních implicitně neuvažujeme.
- Nyní ale chceme analyzovat pomocí grafů **datové struktury**, které přirozeně mohou být prázdné.
- V takovémto kontextu tedy povolujeme i existenci grafu bez vrcholů, který odpovídá prázdné datové struktuře.

Zakořeněné stromy

Definice

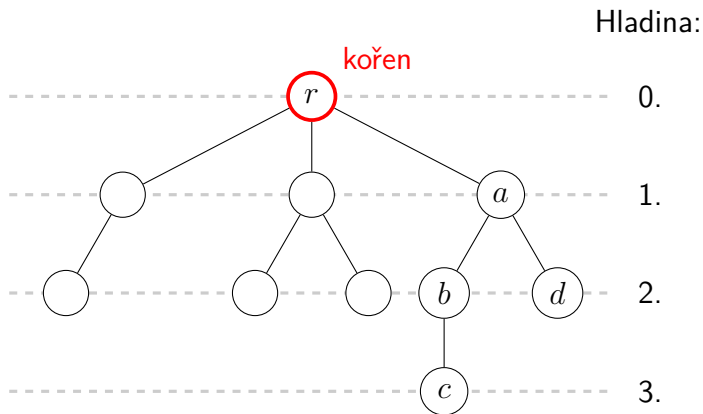
- **Zakořeněný strom** je strom T , ve kterém je jeden vrchol $r \in V(T)$ označen jako **kořen**.
- Leží-li u na (jediné) cestě z v do kořene, pak u je **předek** v a v je **potomek** u . Pokud je navíc $\{u, v\} \in E(T)$ hrana, říkáme, že u je **otec** v a v je **syn** u .
- Vrcholy rozdělíme podle vzdálenosti od kořene do **hladin**: v nulté leží kořen, v první jeho synové, atd.
- **Hloubka zakořeněného stromu** T je počet jeho hladin.

Pozorování

Všechny $v \in V(T) - \{r\}$ jsou potomci kořene r a kořen r je předkem všech ostatních vrcholů.

Připomeňme, že tyto pojmy jsme již zavedli jako BFS hladiny (pokud bychom jej pustili z kořene r).

Zakořeněné stromy – příklad



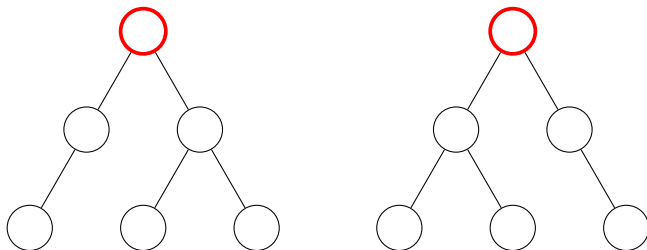
- a je otec b
- b je syn a
- r, a, b jsou předci c
- a, b, c, d jsou potomci r

Binární stromy

Definice

Strom nazveme **binární**, pokud

- je zakořeněný,
- každý vrchol má nejvýše dva syny a
- u synů rozlišujeme, který je **levý** a který **pravý**.



Příklad dvou různých binárních stromů

Binární minimová halda

Definice

Binární minimová halda je datová struktura tvaru binárního stromu, v jehož každém vrcholu x je uložen jeden **klíč** $k(x)$ a která splňuje tyto dvě vlastnosti:

- 1 **Tvar haldy**: Strom má všechny hladiny kromě poslední plně obsazené. Poslední hladina je zaplněna od levého okraje směrem k pravému.
- 2 **Haldové uspořádání**: Je-li v vrchol a s jeho syn, platí

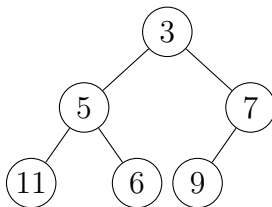
$$k(v) \leq k(s).$$

Poznámky:

- 1) Pokud nebude řečeno jinak, binární minimovou haldu budeme zkráceně označovat **halda**.
- 2) Binární maximová halda je definována analogicky.

Příklad haldy

- 1 **Tvar haldy:** Strom má všechny hladiny kromě poslední plně obsazené. Poslední je zaplněna zleva doprava.
- 2 **Haldové uspořádání:** Je-li v vrchol a s jeho syn, platí $k(v) \leq k(s)$.



Pozorování

Na cestě vedoucí z libovolného vrcholu do kořene tvoří klíče **nerostoucí** posloupnost.

V kořeni je tedy globální minimum ze všech klíčů.

Počet hladin haldy

Lemma

Binární halda s n prvky má $\lfloor \log n \rfloor + 1$ hladin.

Poznámka: nebude-li řečeno jinak, logaritmy budou implicitně dvojkové.

Důkaz.

- Binární strom s $h \geq 1$ úplnými hladinami má $n = 2^0 + 2^1 + 2^2 + \dots + 2^{h-1} = 2^h - 1$ vrcholů.
- Vzhledem ke *Tvaru haldy* přibude nová hladina jen tehdy, když počet prvků n vzroste z $2^h - 1$ na 2^h .
- Z toho plyne vzorec, neboť právě funkce $\lfloor \log n \rfloor$ se právě při této změně n zvětší o jedna a pro $n = 2^h - 1$ je počet hladin $h = (h - 1) + 1 = \lfloor \log(2^h - 1) \rfloor + 1$.



Počet listů haldy

Lemma

Binární halda s $n \geq 3$ prvky má $\lfloor n/2 \rfloor$ vnitřních vrcholů a $\lceil n/2 \rceil$ listů.

Důkaz.

- Indukcí podle počtu vrcholů binárního stromu.
- Lemma platí triviálně pro $n = 3$ (2 listy a kořen – jediný vnitřní vrchol).
- Nechť $n > 3$ je liché. Odebráním posledního listu, který je druhým (tj. pravým) synem posledního vnitřního vrcholu, vznikne halda o sudé velikosti $n - 1$, pro kterou z ind. předp. lemma platí.
- Vrácením posledního listu se počet vnitřních vrcholů nezmění a původní strom jich tedy má $\lfloor (n - 1)/2 \rfloor = \lfloor n/2 \rfloor$.
- A počet listů se zvětší o jedna a původní strom jich tedy má $\lceil (n - 1)/2 \rceil + 1 = \lceil n/2 \rceil$.
- ...

Počet listů haldy

Dokončení důkazu.

- Necht $n \geq 4$ je sudé. Pak poslední list je jediný levý syn svého otce.
- Jeho odebráním vznikne halda liché velikosti $n - 1$, která má podle ind. předpokladu $\lceil (n - 1)/2 \rceil$ listů a $\lfloor (n - 1)/2 \rfloor$ vnitřních vrcholů.
- Vrácením posledního listu jeden vnitřní vrchol přibude a původní strom jich tedy má $\lfloor (n - 1)/2 \rfloor + 1 = \lfloor n/2 \rfloor$.
- A počet listů se nezmění a původní strom jich tedy má $\lceil (n - 1)/2 \rceil = \lceil n/2 \rceil$. □

Binární minimová halda – Formalizace

Binární halda H má atributy

- $H.\text{root}$, který je aktuální kořen
- $H.n$, který udržuje aktuální počet prvků
- $H.\text{last}$, který udržuje ukazatel na aktuální poslední list (nejpravější list v poslední hladině), pokud existuje

Prvek x haldy má

- klíč $k(x)$
- ukazatel na otce
- ukazatele na své syny (jsou-li v haldě obsažené)

Nadále budeme předpokládat, že výše uvedené máme a umíme přepočítat v čase $O(1)$. Na závěr takovou implementaci předvedeme.

Vložení prvku do haldy

- Vlastnost Tvar haldy dovoluje přidat okamžitě nový prvek na konec nejspodnější hladiny.
- Pokud by již byla plná, založíme novou hladinu.
- Pokud je Haldové uspořádání mezi novým listem ℓ a jeho otcem o v pořádku, můžeme skončit.
- Pokud ne, prohodíme $k(\ell)$ a $k(o)$.
- Tím se ale může porušit vlastnost Haldového uspořádání mezi vrcholem o a otcem vrcholu o .
- Pokud pro ně Haldové uspořádání neplatí, opět jejich klíče prohodíme, jinak končíme.
- Toto opakujeme, nejdále však až do kořene.

Algoritmus **HeapInsert**(H, k)

(1) $H.n := H.n + 1$

(2) vlož na první volnou pozici v poslední hladině H
nový vrchol p s klíčem k

(3) **BubbleUp**(H, p)

BubbleUp(H, p)

(4) Dokud $p \neq H.\text{root}$:

(5) $o := \text{otec}(p)$

(6) Pokud $k(o) \leq k(p)$:

(7) return

(8) prohoď $k(o)$ a $k(p)$

(9) $p := o$

Analýza složitosti

Na každé hladině strávíme $O(1)$ operací a procházených hladin je nejvýše logaritmicky mnoho, celkem tedy čas $O(\log n)$.

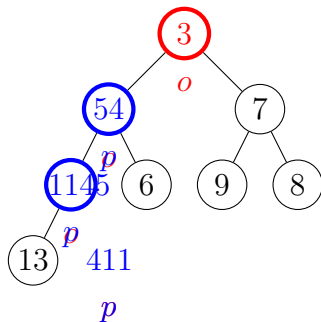
Příklad vložení prvku do haldy

Algoritmus **HeapInsert**(H, k)

- (1) $H.n := H.n + 1$
- (2) vlož na první volnou pozici
v poslední hladině nový
vrchol p s klíčem k
- (3) **BubbleUp**(H, p)

BubbleUp(H, p)

- (4) Dokud $p \neq H.root$:
- (5) $o := otec(p)$
- (6) Pokud $k(o) \leq k(p)$:
- (7) return
- (8) prohoď $k(o)$ a $k(p)$
- (9) $p := o$



Korektnost HeapInsert

Lemma

Bud' H minimová halda a k klíč. Potom struktura vzniklá voláním $\text{HeapInsert}(H, k)$ je opět minimová halda.

Důkaz.

Pokud H je prázdná, pak je výsledkem jednoprvková halda – hotovo. Jinak označme $\hat{H}$ výsledek volání $\text{HeapInsert}(H, k)$. $\hat{H}$ má dle (2) tvar haldy. Zbývá ověřit, že splňuje podmínku haldového uspořádání. Po přidání hodnoty k do nového listu ℓ může existovat jen jedna hrana s porušenou podmínkou. Tento problém pomocí funkce BubbleUp posouváme směrem ke kořeni. Je třeba ověřit, že nemůžeme porušit podmínku haldového uspořádání mezi nějakým otcem o a jeho (případným) druhým synem na cestě z ℓ do kořene.

- Pozorujeme, že hodnota $k(o)$ v o se aplikováním BubbleUp nemohla zvýšit (může klesnout, případně zůstat stejná, pokud otec vrcholu o měl stejnou hodnotu).



Nalezení a odstranění minima

- `HeapFindMin( $H$ )` je triviální: vrať klíč kořene haldy v $O(1)$ čase.
- `HeapExtractMin( $H$ )` je komplikovanější: Odstranění kořene $r = H.\text{root}$ stromu haldy by porušilo vlastnost **tvar haldy**.
- Nicméně je možné triviálně odstranit **poslední list** ℓ , čili list nejvíc vpravo v poslední hladině.
- Z toho plyne idea algoritmu odstranění minima:
 - 1 Prohod' $k(r)$ a $k(\ell)$.
 - 2 Odstraň vrchol ℓ a zmenši haldu o 1 prvek.
 - 3 „Probublávej dolů“ z kořene klíč $k(\ell)$ na jeho správné místo, aby opět platilo haldové uspořádání.

Algoritmus `HeapExtractMin`(H)

- (1) $r = H.\text{root}$, $x = k(r)$, $\ell = H.\text{last}$
- (2) prohoď $k(r)$ a $k(\ell)$
- (3) $H.n := H.n - 1$
- (4) `BubbleDown`(r)
- (5) return x

`BubbleDown`(v)

- (6) Dokud má v nějaké syny:
- (7) $s :=$ takový syn v , který má nejmenší klíč
- (8) Pokud $k(v) \leq k(s)$:
- (9) return
- (10) prohoď $k(v)$ a $k(s)$
- (11) $v := s$

Analýza časové složitosti

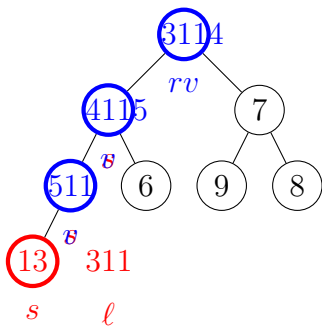
Na každé hladině provedeme $O(1)$ operací a procházených hladin je nejvýše logaritmicky mnoho, tedy celkový čas je $O(\log n)$.

Algoritmus `HeapExtractMin(H)`

- (1) $r = H.\text{root}$, $x = k(r)$, $\ell = H.\text{last}$
- (2) prohoď $k(r)$ a $k(\ell)$
- (3) $H.n := H.n - 1$
- (4) `BubbleDown(r)`
- (5) return x

`BubbleDown(v)`

- (6) Dokud má v nějaké syny:
- (7) $s :=$ takový syn v , který
 má nejmenší klíč
- (8) Pokud $k(v) \leq k(s)$:
- (9) return
- (10) prohoď $k(v)$ a $k(s)$
- (11) $v := s$



Korektnost HeapExtractMin

Lemma

Bud' H minimová halda a k klíč. Potom struktura vzniklá voláním $\text{HeapExtractMin}(H)$ je opět minimová halda.

Důkaz.

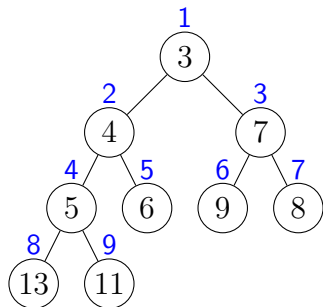
- Tvar haldy je opět zachován triviálně.
- Pokud je výsledkem prázdná či jednoprvková halda, máme hotovo.
- Prohození z (2) mohlo porušit haldové uspořádání (mezi hladinami 0 a 1).
- Opět prohazujeme s menším ze synů, čímž posunujeme porušení na dvojici následujících hladin.
- Toto může pokračovat jen do poslední hladiny. □

K zamyšlení

- Co se změní, když místo minimové haldy budeme pracovat s **maximovou** haldou a místo operací `HeapExtractMin` a `HeapFindMin` budeme tedy podporovat operace `HeapExtractMax` a `HeapFindMax`?
- Rozmyslete algoritmus operace `HeapChangeKey`, která dostane ukazatel na prvek uložený v haldě, ve kterém se má klíč změnit na zadanou hodnotu tak, aby výsledkem byla opět halda.
- Využitím operace `HeapChangeKey` lze definovat operaci, která umožňuje z haldy odstranit libovolný prvek zadaný ukazatelem. Popište její algoritmus.

Jak haldu reprezentovat v paměti

- Reprezentace pomocí dynamicky alokovaných spojových struktur je zbytečně komplikovaná.
- Očíslujme (oindexujeme) vrcholy po hladinách shora dolů a na nich zleva doprava (číslujeme od 1 do n):



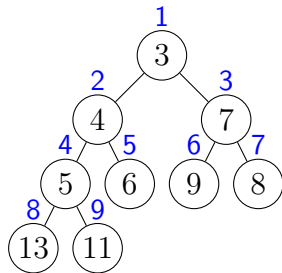
1	2	3	4	5	6	7	8	9
3	4	7	5	6	9	8	13	11

Vztahy indexů v haldě

Pozorování

Má-li vrchol v index i , pak

- jeho levý syn má index $2i$ a
- jeho pravý syn index $2i + 1$,
- jeho otec má index $\lfloor i/2 \rfloor$ a
- výraz $i \bmod 2$ udává, zda je v připojen k otci levou či pravou hranou, tj. zda je jeho levý či pravý syn.



Důsledek

Haldu lze tedy jednoduše reprezentovat v poli a pohyb po stromu realizovat výše popsanými operacemi s indexy.

Rychlá konstrukce haldy - operace HeapBuild

- Haldu z n prvků lze vytvořit z prázdné haldy n voláními operace **HeapInsert**, což dává čas (pomocí Stirlingova vzorce)

$$O(\log 2) + O(\log 3) + \dots + O(\log n) = O(\log(n!)) = O(n \log n).$$

- Jde to však rychleji.
- Idea rychlejšího algoritmu: (1) Má-li prvek v dva syny ℓ a r , kteří jsou kořeny podstromů tvořících již korektní haldy, lze podstrom s kořenem ve v předělat na haldu „zabubláním“ klíče $k(v)$ dolů. (2) Všechny $\lceil n/2 \rceil$ listů jsou 1prvkové haldy.

HeapBuild

Vstup: prvky $x_1, \dots, x_n$

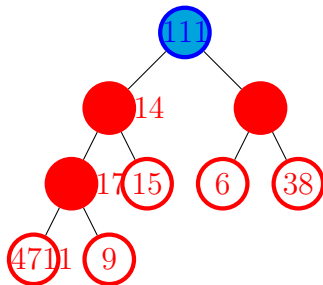
Výstup: Halda $H[1 \dots n]$

(1) vlož prvky do pole H tak, že $H[i] := x_i$

(2) Pro $i := \lfloor n/2 \rfloor, \dots, 1$:

(3) **BubbleDown**($H[i]$)

Příklad vytvoření haldy pomocí HeapBuild



Složitost HeapBuild

Věta

Operace **HeapBuild** má časovou složitost $O(n)$.

Důkaz.

- Nechť strom haldy má h hladin (od 0 do $h - 1$).
- Platí tedy $2^{h-1} \leq n \leq 2^h - 1$.
- Pro účely analýzy si představíme, že i poslední hladina je zcela plná, například nějakými „dummy“ prvky většími než vše ostatní; tím jistě nesnížíme potřebný počet operací.
- Procedura **BubbleDown** od j -té hladiny trvá čas $O(h - 1 - j)$.
- Pokud to sečteme přes všech 2^j vrcholů hladiny j a poté přes všechny hladiny, dostaneme (až na konstantu)

$$\sum_{j=0}^{h-1} 2^j (h - 1 - j) = \sum_{q=0}^{h-1} 2^{h-1-q} q = \sum_{q=0}^{h-1} \frac{2^{h-1}}{2^q} q \leq n \sum_{q=0}^{h-1} \frac{q}{2^q}.$$

Složitost HeapBuild – dokončení důkazu

Věta (d'Alembertovo kritérium – BI-MA2)

Nechť $a_k > 0$ pro každé $k \in \mathbb{N}$. Pokud $\lim_{k \rightarrow \infty} \frac{a_{k+1}}{a_k} < 1$, potom řada $\sum_{k=0}^{\infty} a_k$ absolutně konverguje.

Dokončení důkazu.

Časovou složitost lze shora odhadnout $O(n \sum_{q=0}^{\infty} \frac{q}{2^q}) = O(n)$.

Pro $a_k = \frac{k}{2^k}$ máme:

$$\lim_{k \rightarrow \infty} \left(\frac{k+1}{2^{k+1}} \bigg/ \frac{k}{2^k} \right) = \lim_{k \rightarrow \infty} \frac{(k+1) \cdot 2^k}{k \cdot 2^{k+1}} = \lim_{k \rightarrow \infty} \frac{(k+1)}{k \cdot 2} = \frac{1}{2}.$$

Navíc $k/2^k > 0$, takže jsou splněny předpoklady d'Alembertova kritéria a proto řada $\sum a_k$ konverguje k nějaké kladné reálné konstantě c a tedy: $O(n \sum q/2^q) = O(c \cdot n) = O(n)$. □

Rychlé řazení pomocí haldy – algoritmus HeapSort

- Halda umožňuje zkonstruovat rychlý algoritmus pro řazení.
- Tento algoritmus se nazývá **HeapSort**.
 - ▶ Prvky $x_1, \dots, x_n$ vložíme do pole.
 - ▶ Na toto pole zavoláme **HeapBuild**.
 - ▶ Poté n -krát zavoláme **HeapExtractMin** a vrácené hodnoty vypíšeme do výstupního pole. Tím vygenerujeme vzestupně seřazenou posloupnost.
- Podle předchozích analýz má **HeapSort** časovou složitost $O(n) + O(n \log n) = O(n \log n)$.

K zamyšlení

- Předchozí popis algoritmu **HeapSort** předpokládal, že se odebíraná minima postupně zapisují do výstupního pole.
- Jedná se tedy o efektivnější realizaci myšlenky algoritmu **SelectSort**.
- Algoritmus **HeapSort** lze realizovat **in-place**, tedy bez použití extra paměti na uložení zkonstruované haldy.
- Navrhněte, jak to provést, když vstupní posloupnost je uložena v poli P a kromě tohoto pole smíte použít již jen $O(1)$ pomocných proměnných.

Složitost operací nad dynamickými množinami dat

- Binární minimová halda je příklad **dynamické množiny dat**, jejíž velikost i konkrétní složení se v závislosti na posloupnosti operací **HeapInsert** a **HeapExtractMin** průběžně mění.
- Složitost operace vložení či odebrání prvku takové struktury v **nejhorším** nebo v **nejlepším** případě nevystihuje vždy nejlépe skutečné chování algoritmu takové operace v rámci životního cyklu takové datové struktury.
- Proto je u algoritmů nebo operací, které se volají mnohokrát nad dynamicky se měnící datovou strukturou, užitečné stanovit tzv. **amortizovanou složitost**¹.

¹Naše definice zde je jen pro „zvětšující se“ datové struktury. Preciznější definici zavedeme v BI-AG2.

Amortizovaná časová složitost

Definice

Operace A nad dynamickou datovou strukturou má v daném **kontextu** svého provádění **amortizovanou časovou složitost** $O(f(n))$ (značíme $O^*(f(n))$), pokud posloupnost k operací A má celkovou časovou složitost $O(k \cdot f(n))$. Parametr n je velikost dynamické množiny **po provedení** této posloupnosti operací.

- Pro odhad časové složitosti jednotlivých provedení takové operace se stále bere **nejhorší** případ.
- Amortizovanou složitost je třeba stanovovat přes **dostatečně** dlouhé posloupnosti operací. Typický kontext amortizované analýzy je sekvence všech operací od inicializace datové struktury do vybudování struktury velikosti n .
- Složitost jednoho konkrétního volání operace v nejhorším případě může být **řádově větší**.

Příklad amortizované analýzy: nafukovací pole

Popíšeme si nyní strategii vybudování dynamického 1D pole o velikosti n postupným přidáváním jednotlivých nových prvků tak, že celková časová složitost bude pouze $O(n)$ a tedy přidání jednoho prvku má amortizovanou složitost $O^*(1)$.

Zadání problému nafukovacího pole:

- Na vstup přicházejí průběžně datové prvky, které je potřeba ukládat do pole $P[1, \dots]$, i -tý prvek na index i .
- Navrhněte operaci **NPInsert**, která vkládá prvky do pole, když
 - ★ jejich celkový počet n předem neznáme a ani ho nedokážeme rozumně odhadnout,
 - ★ a tudíž nelze předem alokovat dostatečně velké pole,
 - ★ a na počátku alokujeme pole minimální velikosti $m_0 = 1$.

Strategie vkládání prvků do nafukovacího pole

Zvolíme následující strategii:

- Počáteční kapacitu pole P zvolíme $m_0 = 1$.
- Označme momentální kapacitu pole P v prvcích jako m .
- Předpokládejme, že počet aktuálně uložených prvků v P je $i - 1$ a že chceme vložit i -tý prvek.
- Jestliže $i \leq m$, pak prvek vložíme na volnou pozici i .
- Jestliže $i > m$, pak je v P uloženo m prvků (došla volná místa) a tudíž:
 - ▶ přealokujeme P na velikost $2m$ prvků,
 - ▶ do první poloviny nakopírujeme staré pole,
 - ▶ následně staré pole zrušíme,
 - ▶ nový prvek vložíme na index $m + 1$.

Vkládání prvků do nafukovacího pole

Algoritmus **NPInsert**(P, x)

- (1) Nechť i je pořadové číslo vkládaného prvku od začátku vkládání.
- (2) Pokud $i \leq m$: $P[i] := x$; $i := i + 1$; return
- (3) Pokud $i > m$:
- (4) Alokuj pole P' o velikosti $2m$
- (5) Překopíruj P do první poloviny pole P'
- (6) Dealokuj P ; $P := P'$
- (7) $P[i] := x$; $i := i + 1$

Pozorování

Časová složitost volání **NPInsert** na pole s i prvky je v nejhorším případě $\Theta(i)$.

Amortizovaná analýza nafukovacího pole

Věta

Uvažujme na počátku prázdné nafukovací pole.

Potom celková časová složitost posloupnosti n operací **NPInsert** je $\Theta(n)$, neboli amortizovaná složitost operace **NPInsert** je $\Theta^*(1)$.

Důkaz.

- Práce sestává z vkládání jednotlivých prvků (každý v $O(1)$ čase) proložených zvětšováním pole.
- Celkový čas samotného vkládání je tedy $\Theta(n)$.
- Případné zvětšování při vložení i -tého prvku trvá čas $\Theta(i)$.
- Ke zvětšování dochází právě tehdy, když i je mocnina dvou.
- Všechna zvětšení tedy dohromady stojí $\Theta(2^0 + 2^1 + \dots + 2^k)$, kde 2^k je nejvyšší mocnina dvojky menší než n .
- V závorce je geometrická posloupnost se součtem $2^{k+1} - 1 < 2n$.
- Celková časová složitost proto činí $\Theta(n)$. □

K zamyšlení

- Jak závisí tvrzení předchozí věty na volbě počáteční kapacity m_0 ?
- Jak by se amortizovaná složitost změnila, pokud bychom zvolili jinou strategii nafukování pole? Například, pokud by velikost nového většího pole nebyla volena jako dvojnásobná, ale
 - (a) jako trojnásobná,
 - (b) větší o nějakou pevnou konstantu c a $m_0 = c$.
- Uvažujme na počátku prázdnou binární haldu H a posloupnost n operací `HeapInsert`($H, \dots$). Vypočítejte amortizovanou složitost operace `HeapInsert`.

Binární sčítačka

- **Binární sčítačka** je obvod, který uchovává bitovou reprezentaci čísla uloženou v buňkách nastavitelných na 0 nebo 1.
- Uvažujeme k -bitovou **sériovou sčítačku (ripple-carry)**:
přenos se postupně šíří zprava doleva
- Po zapnutí má sčítačka všechny bity nastaveny na 0.
- Sčítačka podporuje dvě operace:
 - ▶ **Inc**(n) zvýší číslo n reprezentované v sčítačce o 1.
 - ▶ **Add**(n, m) přičte k číslu v sčítačce n binární číslo m .

n :

0	1	0	1
---	---	---	---

n :

1	0	0	1
---	---	---	---

m :

0	0	1	1
---	---	---	---

Inc(n):

0	1	1	0
---	---	---	---

Add(n, m):

1	1	0	0
---	---	---	---

Binární sčítačka: Amortizovaná analýza operace **Inc**

Složitost operace **Inc**(n) = počet **inverzí** bitů v binárním čísle n .

Pozorování

Složitost operace **Inc**(n) k -bitové sériové sčítačky je v nejhorším případě $\leq k$.

$$k = 4 : \quad n : \begin{array}{|c|c|c|c|} \hline 0 & 1 & 1 & 1 \\ \hline \end{array}$$

$$\mathbf{Inc}(n) : \begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 0 \\ \hline \end{array}$$

Amortizovaná analýza binární sčítačky

Věta

Uvažujme na počátku vynulovanou binární sčítačku. Potom celková složitost posloupnosti n volání operace **Inc** měřená počtem bitových inverzí je $O(n)$, neboli amortizovaná složitost **Inc** je $O^*(1)$.

Důkaz: Bankéřova neboli účetní metoda.

- Počet inverzí při jedné operaci **Inc**(n) je od 1 do $\log n$ (závisí na bitovém rozvoji čísla n).
- Každé operaci **Inc** přiřadíme virtuální kredit ve formě 2 mincí $\star$.
- Za každou inverzi zaplatíme 1 mincí.
- ...

Pokračování důkazu.

- Pokud existují nespotřebované mince, bude je **Inc** střídat pro budoucí inverze více bitů do zásoby tak, že na každém jedničkovém bitu si uchová jednu minci.
- Tedy třeba takto:

 * * * * * * * * *

 1 1 0 1 1 1 0 0 0 0 1 1 1 1

- **Inc** vždy invertuje zprava od lsb jedničkové bity na nulové tak dlouho, dokud nenarazí na první nulový bit, který invertuje na jedničkový a tím řetěz inverzí skončí.
- Inverze všech takových jedničkových bitů zaplatíme z nastřádaných mincí * ležících na každém z nich.
- ...

Dokončení důkazu.

- Z kreditu dvou mincí $\star$ přidělených této operaci **Inc** spotřebujeme
 - 1 jednu minci na inverzi první nuly zprava na jedničku a
 - 2 druhou minci $\star$ položíme do zásoby na právě vzniklou jedničku.
- Náš příklad se tedy změní takto:

$$\begin{array}{cccccccccccc} \star\star & \star\star\star & & \star\star\star\star & \longrightarrow & \star\star & \star\star\star & & \star \\ 1101110000 & 1111 & & & & 1101110001 & 0000 \end{array}$$

- Operace **Inc** tudíž zachovává následující invariant:
po skončení každé operace **Inc je na každém jedničkovém bitu čísla sčítačky našetřena jedna $\star$.**
- To znamená, že celkový počet inverzí v posloupnosti n **Inců** provedených za sebou lze zaplatit nejvýše $2n$ mincemi $\star$.
- Čili amortizovaná složitost operace **Inc** je $O^*(1)$. □

Co bychom po dnešku měli znát

- Zakořeněný strom, binární strom.
- Syn, otec, předek, potomek, hladina.
- Binární halda a její operace:
 - ▶ `HeapInsert`,
 - ▶ `HeapFindMin`,
 - ▶ `HeapExtractMin`,
 - ▶ `HeapBuild`.
- Rychlý řadící algoritmus `HeapSort`.
- Amortizovaná analýza.
 - ▶ Nafukovací pole.
 - ▶ Binární sčítačka.